

A PLAIN-LANGUAGE HANDBOOK

How Your Website Works

Everything inside **NELLY DANSEN PRODUCTIONS** — the front end, the back end, the configuration, and what still has to happen before it goes live.

PROJECT	FRAMEWORK	LANGUAGE	STATUS	DATE
nelly-dansen	Next.js 16.2.9	TypeScript	Runs locally · not yet hosted	25 September 2026

CONTENTS

What's in here

Read it start to finish once, then keep it as a reference. Nothing assumes you've written code before.

1	The thirty-second version	What the site actually is
2	Front end and back end	The restaurant, explained properly
3	Your toolkit	Every framework and what it does for you
4	The map of the folders	Where everything lives and why
5	The front end, page by page	Home, Work, Paparazzi, Admin
6	The back end	The four doors and the filing cabinet
7	Two complete journeys	A visitor looking · you uploading
8	Where your content actually lives	Three different places, three rules
9	The configuration files	One by one, in English
10	Hosting	Where it runs now, and how to go live
11	Things worth fixing	Honest notes before launch
12	Cheat sheet & glossary	Commands and jargon

ONE CONVENTION USED THROUGHOUT

You'll see two badges. **SERVER** means the code runs on a computer you control, before the page is sent.

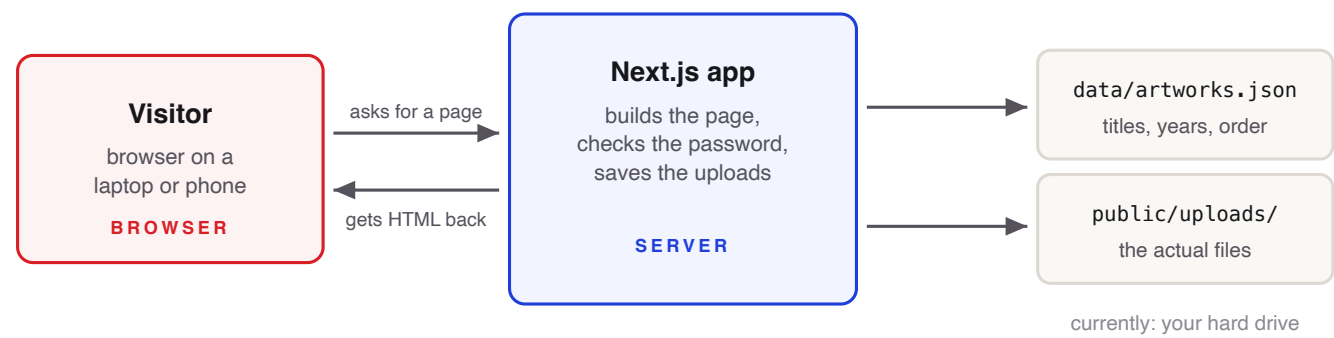
BROWSER means the code runs on the visitor's laptop or phone, after the page arrives. Knowing which is which explains almost every decision in this project.

The thirty-second version

Before any of the detail: here is the whole thing in one paragraph.

Your site is a **small program**, not a set of static pages. When someone visits, the program wakes up, reads your project list out of a file, builds the HTML for that page on the spot, and sends it over. There is also a private door at `/admin`. Behind a password, you can upload images and videos, write titles, crop the thumbnails, and drag projects into the order you want. Those uploads are written straight to disk and the project list file is rewritten. No database, no external service, no monthly bill.

The visual idea sitting on top of that is a **fake desktop**. The home page is a wallpaper — a looping video — with draggable windows floating over it. One window is dressed up as a Google Doc and holds your CV. Another is dressed up as Apple Notes and holds your artist statement and contact details. A red-to-white bar runs across the top as navigation, and a white marquee scrolls your logo across the bottom, forever.



The entire architecture. Three boxes. That's genuinely all of it.

What exists right now

THING	STATE
Four pages	Home, Work, Paparazzi, Admin — all working
Projects uploaded	8 — six under Work, two under Paparazzi
Media files	24 images and videos, about 220 MB
Code on GitHub	Yes — github.com/4296ok/nelly-dansen
Live on the internet	NOT YET — see Section 10

SECTION 2

Front end and back end

These two words get thrown around constantly. Here is what they mean in your project specifically.

The usual analogy is a restaurant: the front of house is what you see, the kitchen is where the work happens. It's a decent start, but it hides the interesting part — in your site, the same programming language builds both, and the same files can contain both. So let's use a sharper distinction.

SERVER The back end

Code that runs **before** anything reaches the visitor, on a computer you control. It can open files, read your password, and write to disk. The visitor never sees this code — only its results.

In your project: everything in `lib/`, everything in `app/api/`, and the top half of each page file.

BROWSER The front end

Code that runs **after** the page has arrived, inside the visitor's browser. It handles anything that reacts to a person — dragging a window, opening a lightbox, typing a password.

In your project: every file whose first line is `"use client"`. There are eight of them.

How the split is marked

In this version of Next.js, **server is the default**. Every component is a server component unless it says otherwise. A file opts into the browser by putting one magic line at the very top:

```
"use client";
```

That line is not decoration. It changes where the code physically executes. A file with it gets bundled up and shipped to the visitor's browser; a file without it never leaves your server. This is why `lib/auth.ts` can safely read your admin password — it lives on the server side of that line and is never sent anywhere.

The eight browser files

`DesktopShell`, `Window`, `windows`, `SiteHeader`, `WorkGallery`, `AdminDashboard`, `LoginForm`, `SquareCropper`. Every one of them needs to respond to a click, a drag, or a keystroke — which is exactly why they're browser files and the rest aren't.

The server files

`app/layout.tsx`, the four `page.tsx` files, all four API routes, and every file in `lib/` except the small helper modules that both sides share (`crop.ts`, `media.ts`, `types.ts` — these are pure calculations with no secrets, so they're safe to use anywhere).

THE RULE OF THUMB

If it needs to know a secret or touch a file, it's back end. If it needs to notice a human moving a mouse, it's front end. Your project follows this cleanly — which is why it's easy to reason about.

The handoff

Server components and browser components aren't separate worlds — they nest. `app/work/page.tsx` is a server component: it reads the project list off disk. It then hands that list to `WorkGallery`, a browser

component, as a plain prop. The data crosses the boundary once, already loaded. The browser never has to ask for it.

```
// app/work/page.tsx – runs on the server
const artworks = await getArtworks("work"); // reads the file
return <WorkGallery artworks={artworks} />; // hands it over
```

This is the pattern used on all four pages, and it's the single most important idea in the codebase.

Your toolkit

Nine pieces of software do the work. Here's what each one is for, in the order you'd meet them.

The three that matter most

1 Next.js 16.2.9 — the framework

The big one. It decides your URLs from your folder names, runs the server, handles the API doors, optimises your fonts, and bundles everything for the browser. Almost every convention in this project is a Next.js convention.

Where you see it: the whole `app/` folder structure, and `next.config.ts`.

2 React 19.2.4 — the interface library

Lets you describe the page as reusable **components** — a Window, a Marquee, a Gallery — instead of one long HTML file. You describe what the screen should look like for a given state, and React works out the changes. When you drag a window, you don't move pixels; you update a number, and React redraws.

Where you see it: every `.tsx` file.

3 TypeScript 5 — the safety net

JavaScript with labels on the data. `lib/types.ts` declares that a project has a title, a year, a category of either "work" or "paparazzi", and a list of images. If any code ever contradicts that, the editor underlines it in red before the site ever runs.

Where you see it: the `.ts` and `.tsx` extensions, and `tsconfig.json`.

The supporting cast

TOOL	JOB	WHAT IT MEANS FOR YOU
Tailwind CSS v4	Styling	Instead of a separate stylesheet, styles are written as short class names right on the element. <code>px-4 py-3 rounded-md</code> means padding across, padding down, rounded corners. Your whole site has just 72 lines of handwritten CSS because of this.
PostCSS	CSS plumbing	The pipe Tailwind runs through. Configured once in <code>postcss.config.mjs</code> and then forgotten about.
Turbopack	Speed	The build engine, written in Rust, built into Next.js 16. It's why <code>npm run dev</code> starts in under half a second and why edits appear on screen instantly.
ESLint 9	Proofreading	Catches mistakes and bad habits automatically. You'll notice comments like <code>eslint-disable-next-line @next/next/no-img-element</code> — that's a deliberate "yes, I know, do it anyway" for plain image tags.

next/font	Typography	Downloads Geist, Geist Mono and EB Garamond <i>at build time</i> and serves them from your own site. No request ever goes to Google when a visitor loads the page — faster, and better for privacy.
npm	Installing	Fetches all of the above. <code>package.json</code> lists what you want; <code>package-lock.json</code> pins the exact versions so it's identical on every machine.

WORTH KNOWING

Your `node_modules` folder holds hundreds of packages, but you only chose the nine above. The rest are things those nine needed. That folder is never committed and never uploaded — it gets rebuilt from `package-lock.json` wherever the site is installed.

The fonts, specifically

Three typefaces are loaded in `app/layout.tsx` and exposed as CSS variables:

- **Geist** (`--font-geist-sans`) — the default for all body text.
- **Geist Mono** (`--font-geist-mono`) — loaded and available, currently unused.
- **EB Garamond** (`--font-garamond`) — the marquee. There's a nice touch here: the marquee asks for *Adobe Garamond Pro* first, in case it's installed on the viewer's machine, and falls back to EB Garamond if not.

SECTION 4

The map of the folders

Ten folders. Once you know what each is for, you can find anything.

app/ ← the pages and the API. Folder names become URLs. **page.tsx** → the home page (/)
layout.tsx → wraps every page: fonts, <html>, page title **globals.css** → the 72 lines of handwritten CSS **favicon.ico** → the little browser-tab icon **work/page.tsx** → /work
paparazzi/page.tsx → /paparazzi **admin/page.tsx** → /admin **api/** ← the back-end doors
auth/route.ts → log in, log out **artworks/route.ts** → list everything, add a project
artworks/[id]/ → edit or delete one project **artworks/reorder/** → save a new running order
components/ ← the reusable visual pieces **SiteHeader.tsx** the red-to-white top bar
WorkGallery.tsx the grid + the lightbox **AdminDashboard.tsx** your whole studio (722 lines – the biggest file) **LoginForm.tsx** the password box **SquareCropper.tsx** drag-to-pan, slide-to-zoom thumbnail framer **desktop/** the fake-desktop illusion **Desktop.tsx** wallpaper video + the construction sign **DesktopShell.tsx** the window manager (which window is on top)
Window.tsx one draggable window + its traffic lights **windows.tsx** the Google-Docs and Apple-Notes costumes **Marquee.tsx** the endless scrolling logo strip **lib/** ← the brains. No visuals here at all. **site.ts** ALL your words: CV, statement, contacts **types.ts** the shape of a project **store.ts** reading and writing projects to disk **auth.ts** the password and the session cookie **crop.ts** the thumbnail-framing maths **media.ts** is this file a photo or a video? **instagram.ts** fetching the live IG feed (currently off) **data/artworks.json** ← your project list. 8 entries today. **public/** ← anything here is downloadable by URL **uploads/** 24 files, ~220 MB – what you added via /admin **bg-mix.mp4** the 7.9 MB wallpaper video **logo.png** the marquee logo **under-construction.png** the sign, bottom-left of the home page **documents/** ← four PDFs. Not wired into the site. **experiments/** ← two standalone HTML sketches. Not part of the app. **WORK/** ← 2 GB of raw source footage. Deliberately excluded.

THE THREE THAT AREN'T REALLY "THE SITE"

documents/, **experiments/** and **WORK/** sit in the folder but aren't part of the running app. **WORK/** is explicitly excluded from version control by a line in **.gitignore** — it would be far too big to carry around. The PDFs and the strobe experiments are simply never imported by anything.

Why **app/** works the way it does

This is Next.js's headline convention, called the **App Router**, and it's worth stating plainly:

- A folder inside **app/** becomes a URL segment.
- A file named **page.tsx** makes that URL a visitable page.
- A file named **route.ts** makes it an API endpoint instead — something code talks to, not people.
- Square brackets mean a variable. **[id]** matches any project ID, so one file serves all eight projects.

So **app/paparazzi/page.tsx** is the page at **/paparazzi**. There is no routing table anywhere. The folders *are* the routing table.

The front end, page by page

Four pages, all built from the same shell.

The shell every page shares

`DesktopShell` is the piece all four pages sit inside. It renders the top bar, and it owns the floating windows. It holds a small amount of memory — for each window, whether it's open, where it is, and how high it stacks. Clicking a window bumps a counter and that window jumps to the front. Dragging updates two numbers and React moves it.

One nice detail: the windows live in an invisible full-screen layer that **ignores clicks** (`pointer-events: none`), and each window switches clicks back on just for itself. That's why a window can float over the gallery without blocking the artwork underneath it.

Another: on a screen narrower than 760 pixels the shell switches to a stacked mobile layout — but only *after* the page has loaded, never during. That ordering is deliberate. Checking the screen width too early would make the server's HTML disagree with the browser's, which React complains about loudly.

Home — /

The fake desktop. A looping `bg-mix.mp4` fills the screen with a dark purple-to-green gradient over the top to keep the windows readable. A still image is named as the poster, so there's something to look at while the video loads. Two windows open by default:

- **The CV window** — styled as a Google Doc, down to the fake address bar reading `docs.google.com/document/d/CV` and a greyed-out "File Edit View Insert Format" menu. Its content comes entirely from `lib/site.ts`.
- **The Notes window** — styled as Apple Notes, holding two notes: your artist statement and your contact list.

The *under construction* sign sits bottom-left, tilted three degrees, and ignores clicks entirely. The code comments note it's meant to be deleted when you're finished.

A THIRD WINDOW EXISTS BUT IS SWITCHED OFF

There's a complete iPhone-shaped window with a notch and an Instagram profile grid inside it. It's disabled by one line in `DesktopShell.tsx`: `const SHOW_CONTACT = false;`. It's waiting on the live Instagram feed (Section 8). Flip that to `true` and it comes back — all the code is intact.

Work — /work

A dark, edge-to-edge grid — two columns on a phone, up to five on a wide screen, with no gaps between tiles. The important structural choice: **one tile per image**, not one tile per project. A project with four images occupies four adjacent tiles, and only the first carries the title overlay. Clicking any tile opens that whole project in a lightbox with a white information panel on the right — title, medium, year, description — and a thumbnail strip along the bottom if there's more than one piece of media.

Paparazzi — /paparazzi

The same component, with two switches thrown. `minimal` removes the title overlays on the tiles and removes the entire white information panel from the lightbox. You get the media on black, and nothing else. It's one line of difference in `app/paparazzi/page.tsx` — a good example of how the codebase reuses rather than duplicates.

Admin — /admin

Your studio, and the most substantial file in the project. Before anything renders, the server checks for a valid session cookie. No cookie, and it returns the password box instead — the dashboard is never sent to an unauthenticated browser at all. With a cookie, you get:

- **Tabs** for Work and Paparazzi, managed separately.
- **An accumulating file picker.** Normally choosing files a second time replaces the first batch. This one keeps them in memory so you can click "Choose images" repeatedly, across different folders, and everything adds up.
- **The square cropper.** Drag the image to pan, slide to zoom. The square outline is exactly what visitors will see.
- **Drag-to-reorder**, plus up and down buttons for precision. The new order saves to the server as soon as you let go.
- **Inline editing** on every project — change text, remove individual images, add more.

How the cropper actually works

This is clever and worth understanding, because it's the kind of thing that's easy to get wrong.

The crop is stored as three numbers: `scale`, `x`, and `y`. Critically, they are **fractions of the frame**, not pixels. So `{ scale: 1.4, x: -0.1, y: 0 }` means "zoom to 140% and shift left by a tenth of the frame's width" — which produces identical framing whether the thumbnail is displayed at 200 pixels or 800. The original file is never modified or re-saved; the crop is applied live as a CSS transform. Reframe it as many times as you like, losing nothing.

`lib/crop.ts` also clamps the values, so you can't drag the image far enough to reveal an empty corner. And if the numbers come out as the default centred crop, it stores nothing at all rather than three redundant numbers.

SECTION 6

The back end

Four API doors and one filing cabinet. That's the entire back end.

The four doors

An API route is a URL that code talks to rather than people. Each one lists which HTTP **methods** it accepts — think of these as verbs: GET means fetch, POST means create, PATCH means change part of, DELETE means remove.

DOOR	VERB	LOCKED?	WHAT IT DOES
/api/auth	POST	—	Checks your password, issues a session cookie
/api/auth	DELETE	—	Logs you out by deleting the cookie
/api/artworks	GET	Public	Returns all projects as JSON
/api/artworks	POST	Yes	Creates a project, saves the uploaded files
/api/artworks/[id]	PATCH	Yes	Edits one project; adds or removes images
/api/artworks/[id]	DELETE	Yes	Deletes a project and its files
/api/artworks/reorder	POST	Yes	Saves a new running order

Every locked door opens with the same two lines, the first thing in the function:

```
if (!(await isAuthenticated())) {  
  return NextResponse.json({ error: "Not authorized" }, { status: 401 });  
}
```

WHY THIS MATTERS MORE THAN HIDING THE ADMIN PAGE

Hiding a page only stops people who use a browser. These checks live on the *doors*, so someone sending a request directly with a tool — bypassing your interface entirely — still gets turned away. This is the correct place to put the lock, and it's done consistently on all four write operations.

How the password works

1 The password lives in a file that never leaves your machine

`.env.local` holds one line: `ADMIN_PASSWORD=...`. The `.gitignore` excludes every `.env` file, so it's never committed to GitHub and never uploaded anywhere.

2 You type it once at /admin

The login form sends it to `/api/auth`. The server compares it to the environment variable.

3

You get a cookie — but not your password back

This is the good bit. The server runs your password through SHA-256, a one-way scramble, takes the first 32 characters, and stores *that* in the cookie. Your actual password is never in your browser. And because the token is derived from the password, changing the password instantly invalidates every old session.

4

The cookie is locked down

It's set `httpOnly`, meaning no JavaScript on the page can read it — a stray script can't steal your session. It's `sameSite: lax`, so another website can't make your browser use it. In production it's `secure`, so it only travels over HTTPS. It lasts 30 days.

The filing cabinet — `lib/store.ts`

Every read and write of a project goes through this one file. It's the only thing in the codebase that knows where projects are actually kept, and that is a deliberate design decision — it's what makes Section 10's move to a real database a contained job rather than a rewrite.

FUNCTION	JOB
<code>getArtworks(category?)</code>	Read all projects, optionally filter to one gallery, sort by order
<code>getArtwork(id)</code>	Find one
<code>addArtwork(input)</code>	Save the files, add the entry, put it first
<code>updateArtwork(id, input)</code>	Change fields, add images, delete removed ones
<code>reorderArtworks(ids)</code>	Rewrite the order numbers from a list
<code>deleteArtwork(id)</code>	Remove the entry and unlink every file

Three details in here that show care

- **Old records still work.** An earlier version stored a single `image`; this one stores a list called `images`. Every read passes through a `normalize` step that quietly converts the old shape to the new one. Nothing breaks.
- **New work goes to the top.** `addArtwork` finds the lowest order number in use and goes one lower — which is why your order values are all negative. A small trick, but it means new uploads appear first without renumbering everything.
- **Deleting files is best-effort.** If an image file is already missing, the deletion is wrapped in a `try` that swallows the error. A missing file shouldn't stop you removing a project.

Why every page says `force-dynamic`

All four pages carry this line:

```
export const dynamic = "force-dynamic";
```

By default Next.js is aggressive about caching — it would happily build your gallery once and serve that same copy to everyone for a long time. This line switches that off. Every visit re-reads `data/artworks.json` from disk. Slightly more work per visit, but it means the moment you publish something in `/admin`, it's live. For a portfolio with a handful of visitors, that's the right trade.

Two complete journeys

Follow a single request all the way through, twice. This is where the pieces connect.

Journey one — someone opens /work

- 1 The request arrives** **SERVER**
Next.js matches the URL `/work` to the folder `app/work/` and runs its `page.tsx`.
- 2 The page fetches its data** **SERVER**
It calls two things at once — `getArtworks("work")` and `getInstagramFeed()` — using `Promise.all`, so they run in parallel instead of one waiting for the other.
- 3 The store reads the file** **SERVER**
`getArtworks` opens `data/artworks.json`, parses it, normalises every record, keeps only the six with category `"work"`, and sorts them by their order number.
- 4 The gallery is described** **SERVER**
That list is handed to `WorkGallery`, which flattens it: six projects containing seventeen images become seventeen tiles, with each project's images kept adjacent and the title riding on the first.
- 5 HTML is sent** **SERVER**
Next.js turns all of that into finished HTML and sends it. The visitor sees the artwork immediately — before any JavaScript has even downloaded.
- 6 The browser takes over** **BROWSER**
React's code arrives and attaches to the HTML already on screen — a process called **hydration**. Only now do the tiles become clickable and the windows draggable.
- 7 A tile is clicked** **BROWSER**
The lightbox opens instantly. No request to the server — every project's full details were already in the page from step four.

Journey two — you upload a new project

- 1 You open /admin** **SERVER**
The server checks your cookie before rendering anything. Valid, so it reads all eight projects and renders the dashboard with them already loaded.
- 2 You pick files and frame the cover** **BROWSER**
Your files are held in the browser's memory — nothing has been sent yet. The cropper shows a live preview and records three numbers.

- 3 You press upload** **BROWSER**
The dashboard packs the text fields, the crop numbers, the category and every file into a `FormData` bundle and POSTs it to `/api/artworks`.
- 4 The door checks you, then checks the files** **SERVER**
Session cookie first. Then: at least one file present, and every file's type starts with `image/` or `video/`. Anything else is rejected before a single byte is written to disk.
- 5 The files are written** **SERVER**
A random ID is generated. Each file is saved as `{id}-{number}.{ext}` into `public/uploads/`. Renaming this way means two files called `IMG_4471.jpg` can never collide.
- 6 The list is rewritten** **SERVER**
A new entry is appended with your text, the image paths, the crop, and an order number one lower than the current minimum. The whole `artworks.json` is written back out.
- 7 It's live** **BROWSER**
The dashboard adds the new project to its own list without reloading. And because `/work` is `force-dynamic`, the very next visitor gets the updated gallery.

NOTICE WHAT ISN'T IN EITHER JOURNEY

No database. No image CDN. No third-party service. No build step between you pressing upload and a visitor seeing it. That simplicity is the reason the site costs nothing and has nothing to break — and it's also exactly what Section 10 has to change.

SECTION 8

Where your content actually lives

Three separate places, with three different rules for changing them. Getting these straight will save you a lot of confusion.

PLACE	HOLDS	HOW YOU CHANGE IT
<code>lib/site.ts</code>	CV, artist statement, contacts, site title, marquee text, wallpaper filename	Edit the file. Requires a developer (or Claude). Not editable from <code>/admin</code> .
<code>data/artworks.json</code>	The eight projects — titles, mediums, years, descriptions, image paths, crops, order	Entirely through <code>/admin</code> . Never edit this by hand.
<code>public/uploads/</code>	The 24 actual image and video files	Entirely through <code>/admin</code> .

THE ONE THING TO REMEMBER

Your **CV and artist statement are code**, not content. They're a long list inside `lib/site.ts`. Adding a new exhibition means editing that file, not logging into the admin. If you want to change that — and for a CV that updates as often as yours does, you probably should — it's a well-defined piece of work: give the CV the same treatment the artworks already have.

The Instagram feed

`lib/instagram.ts` is complete and ready, and currently doing nothing. It's designed to work with **behold.so**, a free service that mirrors an Instagram account's recent posts. To switch it on:

1. Sign up at behold.so and connect @nellydansen.
2. Create a feed and copy its ID.
3. Set `BEHOLD_FEED_ID` as an environment variable, or paste the ID into `site.contact.instagramFeed`.
4. Change `SHOW_CONTACT` to `true` in `DesktopShell.tsx` to bring the phone window back.

It's written defensively: if the request fails or no ID is set, it returns an empty feed rather than crashing, and the interface falls back gracefully. It also caches for ten minutes so behold isn't hit on every single page view.

How images and videos are told apart

There's no separate field marking a file as a video. `lib/media.ts` simply checks the extension against a list — `.mp4`, `.webm`, `.mov`, `.m4v`, `.ogg`, `.ogv`. If it matches, the gallery renders a `<video>` element (muted, looping, autoplaying in the grid; with controls in the lightbox). If not, an image. Simple, and it means videos needed no changes to the data format at all.

SECTION 9

The configuration files

Eight files in the root folder that nobody ever looks at. Here's what each is for.

`package.json`

The project's identity card: its name, the nine tools it depends on, and four commands you can run. The commands are `dev` (start the development server), `build` (prepare for production), `start` (run the built version), and `lint` (proofread the code).

`package-lock.json`

The exact version of every single package, including the hundreds pulled in indirectly. This is what guarantees the site behaves identically on your laptop and on a hosting server. Never edit it; npm maintains it.

`next.config.ts`

Almost empty, which is a good sign. It contains one setting: pinning Turbopack's root folder to this project, so it doesn't get confused by other Node projects elsewhere on your machine. That's it.

`tsconfig.json`

TypeScript's settings. Two lines matter to you: `"strict": true`, which turns on all the safety checks, and the `paths` entry mapping `@/` to the project root. That's why files say `@/lib/site` rather than `../../lib/site` — it's the same thing, just readable.

`postcss.config.mjs` and `app/globals.css`

The first hooks Tailwind into the styling pipeline. The second is your only handwritten CSS — 72 lines. Worth knowing what's in it, because it's where the site's personality lives:

- **Colour variables**, with a full dark-mode set that automatically follows the visitor's system setting.
- `@keyframes marquee` — slides a strip left by exactly 50% over 30 seconds, forever. Because the logo group is rendered twice, that lands precisely where it started, which is what makes the loop seamless.
- `.title-glow` — the red neon on the site title, built from three stacked text shadows.
- `.bevel-emboss` — the raised 3D effect on the nav links: one white shadow up-and-left, one black shadow down-and-right. A deliberate late-90s reference.
- `.masonry` — column styles, currently unused by any page.

`eslint.config.mjs`

Proofreading rules, extending the standard Next.js set with its accessibility and performance checks.

`.gitignore`

What version control should ignore: `node_modules`, build output, every `.env` file, and `WORK/`. It also carries a note that's worth repeating, because it's an unusual and temporary decision:

Your uploads and `artworks.json` are committed to GitHub right now — normally they wouldn't be. That's deliberate, so a preview deployment has real work to show. The file itself notes this changes when you move to a real database.

`.env.local`

One line: your admin password. Never committed, never uploaded. If this file goes missing, the code falls back to the password `changeme` — see Section 11.

Hosting

You asked how it's hosted. The honest answer is the most useful one.

CURRENT STATUS

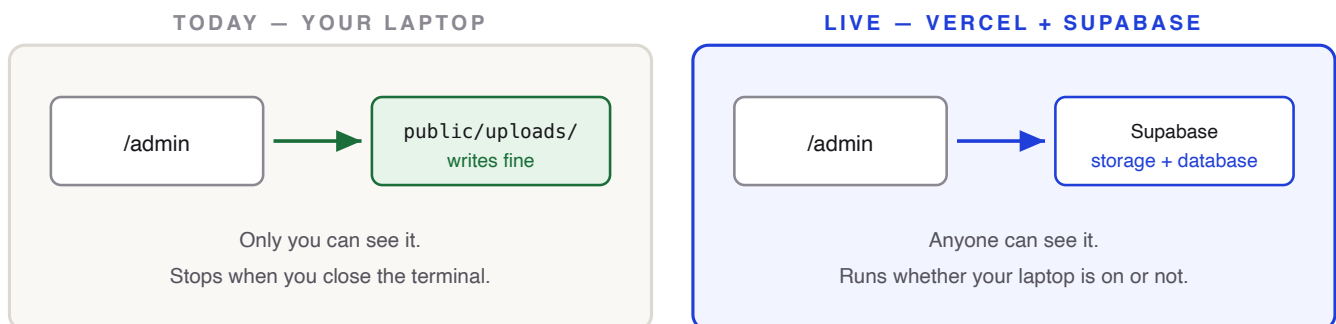
The site is not hosted anywhere yet. It runs only on your laptop, at `localhost:3000`, and only while `npm run dev` is running. There's no Vercel project linked to this folder, no deployment configuration, and no Vercel CLI installed. The code is backed up on GitHub at `github.com/4296ok/nelly-dansen` — but GitHub stores code, it doesn't serve websites.

This is a completely normal place for a project to be. Nothing is wrong. There's just one real obstacle between here and a live site, and it's worth understanding properly before you spend any money or time.

The obstacle

Hosting platforms like Vercel run your code on servers with a **read-only filesystem**. Your code can read files, but it cannot write new ones — and even if it could, those servers are replaced constantly, so anything written would vanish within hours.

Your admin area works by writing files. Uploading saves images to `public/uploads/`; every edit rewrites `data/artworks.json`. Deploy as-is, and the public site displays perfectly — but the moment you try to upload something, it fails.



The only structural change needed: where files get written.

THE GOOD NEWS

This was anticipated. Every read and write goes through `lib/store.ts` and nothing else. Swapping the storage means rewriting the insides of six functions in one file. Not a single page, component, or API route has to change — they'll keep calling `getArtworks()` and `addArtwork()` exactly as they do now.

Going live, step by step

1 Create two free accounts

Vercel (`vercel.com`) runs your code — sign in with GitHub so it can see your repository. Supabase (`supabase.com`) stores your data and files. Both have free tiers that comfortably cover a portfolio.

- 2 Set up Supabase**

Create a public **storage bucket** called `artworks` for the media, and a **table** called `artworks` whose columns match `lib/types.ts`. Copy the project URL and API keys.
- 3 Rewrite lib/store.ts**

Install `@supabase/supabase-js`, then replace the insides of the six functions so they read and write the Supabase table, and upload files to the bucket instead of to disk. This is the only real coding work in the whole list. Claude can do it — the README even suggests the exact phrasing.
- 4 Move the existing work across**

Your eight projects and 24 files need to go into Supabase once. A small one-off script that reads `artworks.json` and pushes each entry and file up. Don't do this by hand.
- 5 Deal with the 220 MB**

Once the media lives in Supabase, remove `public/uploads/` from the repository and restore the normal `.gitignore` rule. Carrying 220 MB of media in a code repository makes every deploy slow and will eventually cause problems.
- 6 Deploy**

In Vercel: **New Project** → **import nelly-dansen** → **Deploy**. Add `ADMIN_PASSWORD` and the Supabase keys under Environment Variables before the first build. You'll get a URL like `nelly-dansen.vercel.app` within a couple of minutes.
- 7 Point a domain at it**

Buy a domain wherever you like, add it in Vercel's Domains panel, and follow the DNS instructions. HTTPS is issued automatically and free.
- 8 Remove the scaffolding**

Delete the under-construction sign from `Desktop.tsx`. Decide on the Instagram window. Fill in the placeholder values still sitting in `site.contact` (it currently says `you@example.com`).

After it's live

Deployment becomes automatic. Push to the `main` branch and Vercel rebuilds and publishes within a minute or two. Push to a branch like your existing `feature/paparazzi-crop-video-reorder` and you get a private preview URL to check before merging. You never run a deploy command.

Adding artwork won't involve any of that — you'll log into `/admin` on the live site, upload, and it appears immediately. That's the entire point of the admin area.

Things worth fixing

Nothing here is urgent while the site is on your laptop. All of it matters before it's public.

Before launch

ISSUE	WHY, AND WHAT TO DO
The fallback password	If <code>ADMIN_PASSWORD</code> isn't set, <code>lib/auth.ts</code> silently falls back to <code>"changeme"</code> . Convenient locally; a serious hole if you ever deploy without setting the variable. It should refuse to start instead. Make sure the variable is set in Vercel <i>before</i> the first deploy either way.
No limit on login attempts	<code>/api/auth</code> will accept unlimited guesses at any speed. Pick a long password, and consider adding a simple rate limit.
220 MB in the repository	Explained in Section 10, step 5. It makes every clone and deploy slow, and it'll only grow.
Placeholder contact details	<code>site.contact</code> still contains <code>you@example.com</code> and <code>"Your short bio line here"</code> . Currently invisible because the phone window is switched off — but it would appear the moment you turn it on.
Two projects called "Untitled"	Both Paparazzi entries. Fine if intentional — Paparazzi doesn't display titles — but worth a conscious decision.

Smaller things

- **The README is out of date.** It describes an `/about` page and individual artwork pages that don't exist, and says uploads are git-ignored, which is no longer true. Worth a rewrite so it doesn't mislead you later.
- **Password comparison isn't constant-time.** `verifyPassword` uses a plain `===`, which in theory leaks a tiny amount of timing information. Genuinely low risk here, but `crypto.timingSafeEqual` is the correct tool and is a two-line change.
- **Images bypass Next.js optimisation.** The galleries use plain `<img>` tags with the linter deliberately silenced. That's a reasonable choice for an art site where you want the exact file shown — but it does mean a visitor on a phone downloads full-size images. Worth revisiting if the site feels slow on mobile.
- **A branch is still open.** `feature/paparazzi-crop-video-reorder` exists locally and on GitHub alongside `main`. Tidy it up if its work is already merged.
- **The under-construction sign** is still on the home page. Its own comment says to delete it when finished.

What's genuinely well built

For balance, because most of this codebase is in good shape:

- Authorisation is checked on **every** write endpoint, without exception.
- Upload types are validated on the server, not just in the browser.
- All untrusted input — crop values, category names — passes through coercion functions that can't be tricked into producing a bad value.

- Storage is isolated behind one module, which is what makes the Supabase migration a contained job.
- Old data formats are handled gracefully rather than breaking.
- The Instagram integration fails softly instead of taking the page down with it.
- The comments explain *why*, not *what* — which is the hard kind to write and the useful kind to read.

Cheat sheet & glossary

Commands

TYPE THIS	WHAT HAPPENS
<code>npm install</code>	Downloads all the tools. Needed once, and after changing dependencies.
<code>npm run dev</code>	Starts the site at <code>localhost:3000</code> . Leave it running while you work — edits appear instantly.
<code>npm run build</code>	Makes the optimised production version. Good for checking nothing is broken.
<code>npm run lint</code>	Proofreads the code.
<code>Control + C</code>	Stops the running server.

Addresses

<code>localhost:3000</code>	Home — the fake desktop
<code>localhost:3000/work</code>	The main gallery
<code>localhost:3000/paparazzi</code>	The Paparazzi gallery
<code>localhost:3000/admin</code>	Your studio (password required)

Glossary

WORD	WHAT IT MEANS HERE
API	A URL that code talks to instead of people. Yours are all under <code>/api/</code> .
Build	Turning your readable source code into the compressed version a browser runs.
Client	The visitor's browser. "Client-side" = runs on their machine.
Component	One reusable piece of interface — a window, a marquee, a gallery.
Cookie	A small note the server asks the browser to keep and show on each visit. Yours proves you're logged in.
Deploy	Publishing your code so the public can reach it.
Environment variable	A setting kept outside the code, so secrets never get committed. Yours is the admin password.
Hydration	The moment React attaches to server-rendered HTML and makes it interactive.
JSON	A plain-text way of writing structured data. Your project list is JSON.

Props	The values passed into a component — like <code>artworks</code> being handed to the gallery.
Repository (repo)	A project tracked by Git, with its full history. Yours is on GitHub.
Route	One URL the site knows how to answer.
Server	The computer running your code before the page reaches anyone.
State	What a component currently remembers — which window is open, which image is showing.

IF YOU REMEMBER THREE THINGS

One. Your words live in `lib/site.ts`; your artwork lives in `data/artworks.json` and `public/uploads/`. Different rules for each.

Two. `"use client"` at the top of a file is the line between the server and the browser, and it explains almost every structural decision in the project.

Three. The site isn't live yet, and the one thing standing in the way is that hosting servers can't write files. Rewriting `lib/store.ts` to use Supabase solves it, and nothing else has to change.